

Universidad Nacional Autónoma de México



Facultad de Ingeniería

Laboratorio de Arquitectura y Organización de Computadoras

Grupo: 07 - Semestre: 2027-1

Reporte Práctica 7.

Procesador CISC 68HC11

Fecha de Entrega

13 de Octubre del 2026

Profesor

Ing. Julio Cesar Cruz Estrada

Integrantes:

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

1. Objetivo

Diseñar un procesador CISC de 8 bits, específicamente un `clon` del miniprocesador 68HC11 de Motorola ©.

2. Desarrollo

En esta práctica se proporcionó un proyecto de Quartus el cual contenía la arquitectura CISC del procesador `M68Hc11`, el que ya contaba con el ciclo de búsqueda y con las instrucciones `LDAB` y `LDAA` en modo inmediato, `ABA` en modo inherente y `JMP` en modo extendido. A partir de este proyecto se agregaron seis rutinas de microprograma: `LDAB` (INM) con el código `D6`, `CMP` (INH), `JE` (EXT), `STAA` (EXT), `LDAA` (EXT) y `LDAB` (EXT). A partir de la implementación de estas instrucciones se implementaron tres programas que se cargaron en la memoria RAM mediante el archivo `mem_content.mif`.

2.1. Arquitectura del procesador

Para la implementación de las rutinas requeridas se empleo como referencia la arquitectura del procesador CISC 68HC11, que se muestra en la Figura 1. Donde se puede observar que la arquitectura cuenta con dos acumuladores de 8 bits (A y B), una unidad de procesamiento aritmético (UPA) y un registro de códigos de condición (CCR) para almacenar las banderas. Los registros de 16 bits (X, Y, registro auxiliar RA, apuntador de pila y contador de programa PC) se conectan a los buses internos C (parte alta), D (parte baja) y E (bus de direcciones). La memoria externa se accede a través del registro de direcciones, que se carga con la señal $\overline{\text{CBD}}$.

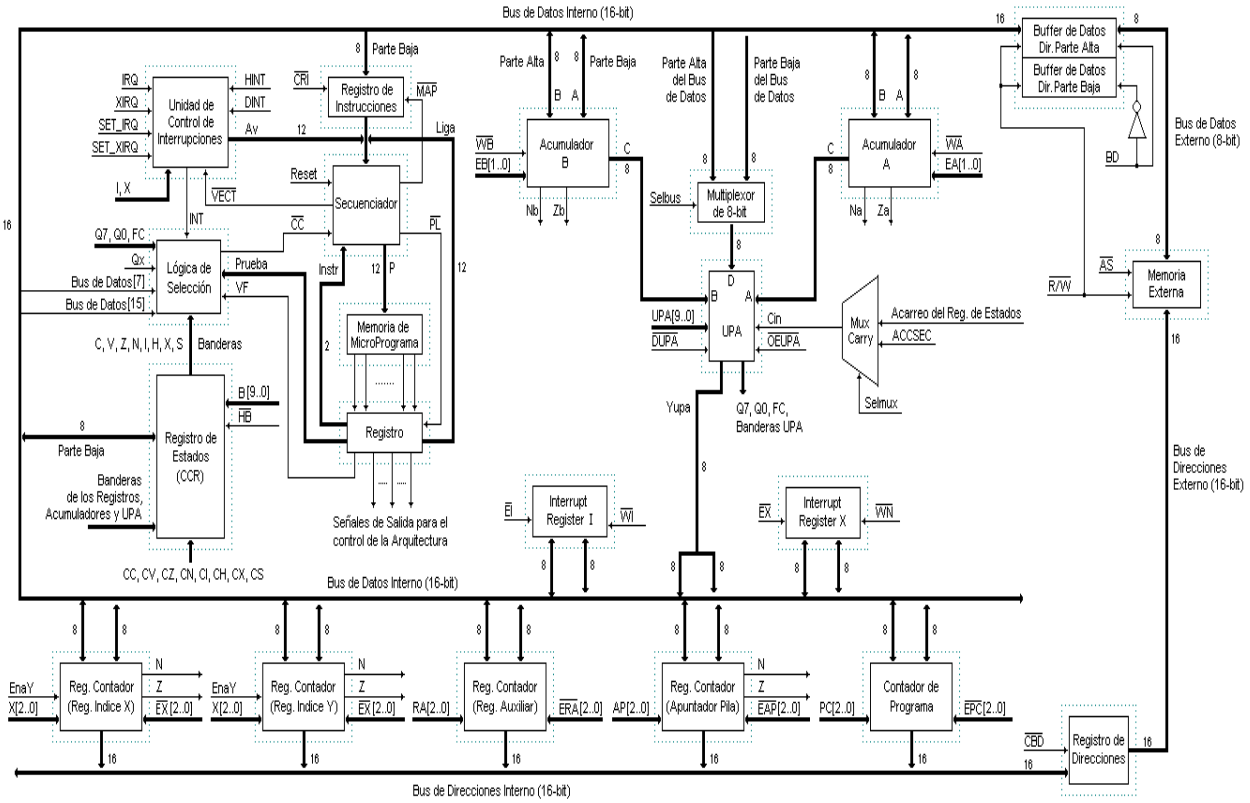


Figura 1: Arquitectura del procesador CISC 68HC11 empleada en la práctica.

Donde el registro de estado presente (`Edo_Pres`) direcciona la memoria de microprograma (`memory.vhd`), la palabra leída se captura en el registro de microinstrucción (`registro_sec`) donde en cada flanco de reloj las señales de control asociadas a un estado se ejecutan durante el ciclo siguiente. La lógica de selección compara la entrada elegida por el campo de prueba con el bit VF mediante una compuerta XNOR:

$$CC = \overline{\text{Prueba} \oplus VF},$$

donde el bloque `logica_seleccion.vhd` interpreta los dos bits del campo de instrucción, según la Tabla[1].

Tabla 1: Códigos de instrucción del secuenciador.

$I_1 I_0$	Nombre	Siguiente estado
`00`	Continuar	$N + 1$.
`01`	Salto condicional	Liga si $CC = 1$; $N + 1$ si $CC = 0$.
`10`	MAP (decodificación)	Dirección formada por el opcode concatenado con `0`.
`11`	Vector	Rutina de interrupción si $CC = 1$; $N + 1$ si $CC = 0$.

2.2. Formato y dimensionamiento de la memoria de microprograma

A partir de la arquitectura del procesador se identificó el formato de la palabra de microprograma y se dimensionó la memoria.

Donde cada palabra de la memoria de microprograma tiene la siguiente organización:

$$\underbrace{P_4 \ldots P_0}_{\text{prueba}}\ \underbrace{VF}_{\text{valor}}\ \underbrace{I_1\, I_0}_{\text{instrucción}}\ \underbrace{L_{11} \ldots L_0}_{\text{liga}}\ \underbrace{\overline{\text{CRI}} \ldots \text{ACCSEC}}_{\text{salidas de control}}.$$

El número de bits por palabra es

$$m = 5 + 1 + 2 + 12 + 78 = 98 \text{ bits},$$

y como la dirección del secuenciador tiene 12 bits, el espacio de la memoria es

$$2^{12} \times 98 = 4096 \times 98 = 401\,408 \text{ bits}.$$

De forma que en el archivo `memory.vhd` sólo se describen las microdirecciones utilizadas, donde cualquier otra dirección entrega la palabra por omisión, la cuál es todas las salidas inactivas.

2.3. Ciclo de búsqueda y decodificación

Para la implementación de las rutinas se tomo como referencia la Figura [2], en la cuál muestra la carta ASM del ciclo de búsqueda y de las instrucciones en modo inmediato e inherente. En `008H` el PC se copia al registro de direcciones, en `009H` se lee el opcode, se guarda en el registro de instrucciones y se incrementa el PC, mientras que en `00AH` la instrucción MAP salta a la dirección opcode $\|$ `0`. Por ejemplo, el opcode `D6` conduce a `D60H` y `1C` a `1C0H`. Al terminar, cada rutina regresa a `009H` después de cargar nuevamente el registro de direcciones con el PC.

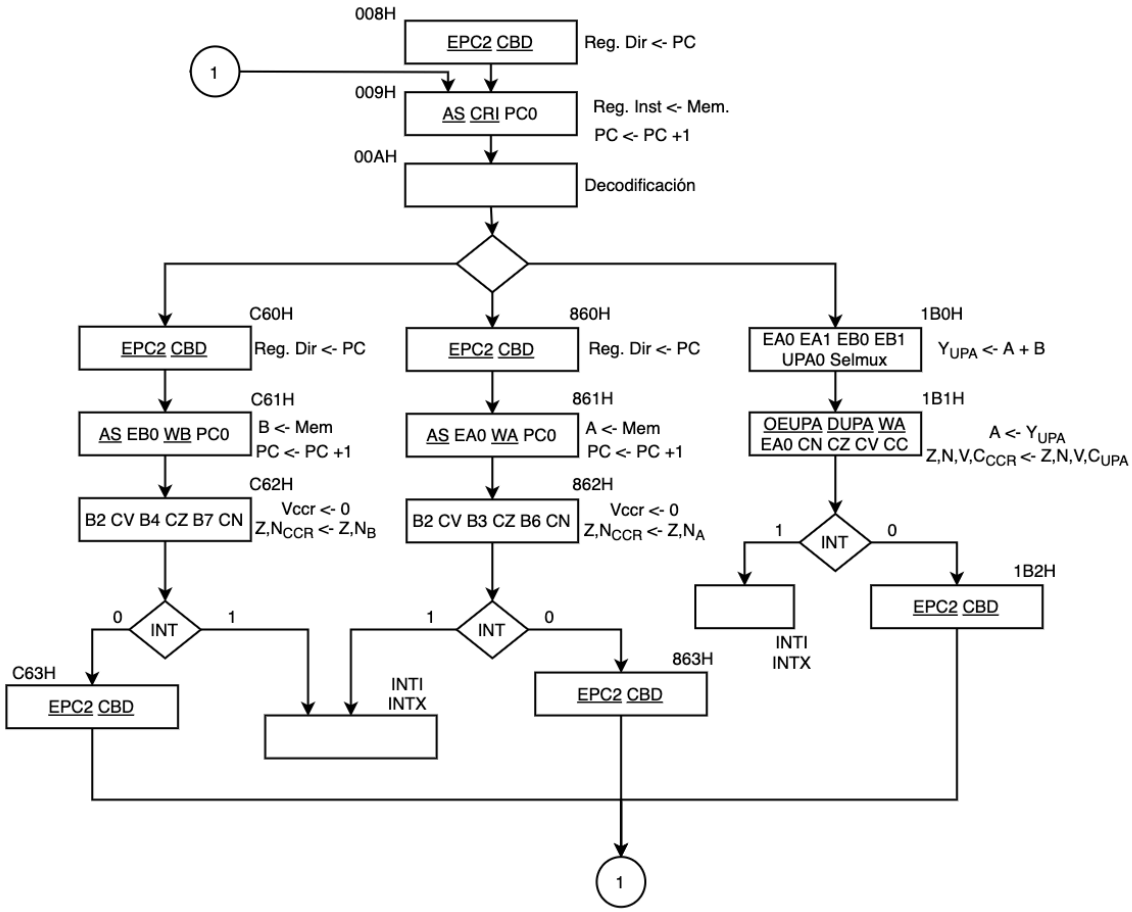


Figura 2: Carta ASM del ciclo de búsqueda y de las instrucciones `LDAB` (INM), `LDAA` (INM) y `ABA` (INH).

Otra carta ASM que se uso de referencia, fue la Figura 3 la cual muestra la carta ASM del modo extendido para `JMP` y `STAA`. En ambos casos los dos bytes que siguen al opcode se leen en el registro auxiliar. Primero la parte alta (con $BD = 1$) y después la parte baja, en cada lectura se incrementa el PC. Con la dirección completa en `RA`, `JMP` la copia al PC y `STAA` la usa como dirección de escritura. Cabe señalar que, aunque el diagrama de referencia anota $Y_{\text{UPA}} \leftarrow A \oplus 0$ en `7E0H`, las señales activas (`nEPC2`, `nCBD`) ejecutan la transferencia Reg. Dir $\leftarrow$ PC, tal como se implementa en `memory.vhd`.

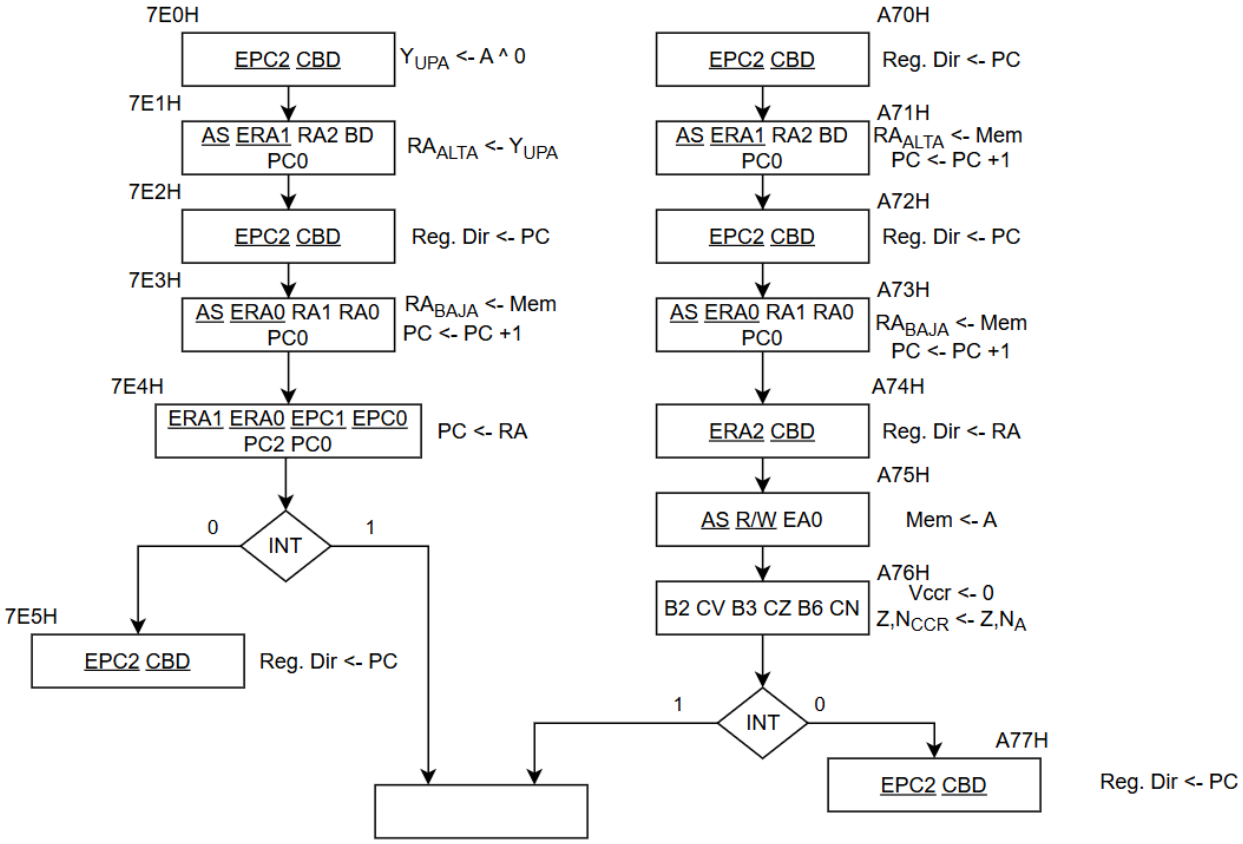


Figura 3: Carta ASM de las instrucciones `JMP` (EXT) y `STAA` (EXT).

2.4. Procedimiento para microprogramar las instrucciones

Para la implementación de las instrucciones se empleo la siguente metología, a partir de una hoja de cálculo junto con la carta ASM de referencia, se fueron colocando los valores correspondientes a cada palabra de control en la memoria de microprograma

Por lo que la metodología para implementar cada instrucción fue la siguiente:

1. Identificar en la carta ASM las transferencias de cada estado.
2. Copiar las filas de una instrucción ya implementada cuya secuencia fuera parecida.
3. Cambiar únicamente las salidas necesarias como salidas activas, escrituras, función de la UPA, banderas y, en su caso, los campos de prueba, VF, instrucción y liga.
4. Copiar la cadena resultante de concatenar los valores de cada fila al archivo `memory.vhd`, en una condición `elsif` con la microdirección correspondiente.

Las Tablas[2] a [6] muestran, para cada estado, la transferencia realizada y las señales que son diferentes de la palabra cuando se ejecuta la instrucción. Donde las señales con prefijo `n` indican que son activas en bajo. La columna *Sec.* indica los campos de secuenciamiento como `prueba/VF/instrucción/liga`, por otro lado el guion (`-`) indica `00000/0/00/000`.

2.4.1. LDAB (INM), opcode `D6`

Para la rutina `LDAB (INM)` se consideró que la carga inmediata de B ya existía en las microdirecciones `C60`–`C63`. Se definió para esta rutina el opcode `D6`, por lo que se copiaron las cuatro filas a `D60`–`D63` sin modificar ninguna salida. El estado `D62` actualiza las banderas con los valores de B ($B7$ y $B4$ seleccionan N_B y Z_B) y $B2$ limpia V.

Tabla 2: Microprograma de `LDAB` (INM) en `D60`–`D63`.

Dir.	Transferencia	Señales activas	Sec.
`D60`	Reg. Dir ← PC	nEPC2=0, nCBD=0	–
`D61`	$B \leftarrow$ Mem, PC ← PC+1	EB0=1, nWB=0, PC0=1, nAS=0	–
`D62`	$V \leftarrow 0$; $Z, N \leftarrow Z_B, N_B$	B7, B4, B2, CN, CV, CZ	`01111/1/11/000`
`D63`	Reg. Dir ← PC	nEPC2=0, nCBD=0	`11000/0/01/009`

2.4.2. CMP (INH), opcode `1C`

La instrucción `CMP` realiza $A - B$ para actualizar las banderas, sin embargo no guarda el resultado. Se tomó como referencia la instrucción `ABA` (`1B0`–`1B2`), dado que ambas conectan los dos acumuladores a la UPA. Los cambios que se realizaron fueron los siguientes:

- En `1C0` se agregó `UPA5=1`, que cambia la función de la UPA de suma a resta $R - S - \overline{C_{in}}$ con $R = A$ y $S = B$, y `ACCSEC=1`, que selecciona el acarreo personalizado para que el resultado sea $A - B$.
- En `1C1` se eliminaron `EA0=1` y `nWA=0`, de modo que la salida de la UPA no se escribe en A. También se quitó `CH`, porque la comparación sólo actualiza C, N, V y Z.
- `1C2` es igual a `1B2`.

Tabla 3: Microprograma de `CMP` (INH) en `1C0`–`1C2`.

Dir.	Transferencia	Señales activas	Sec.
`1C0`	$Y_{UPA} \leftarrow A - B$	EB1, EB0, EA1, EA0, **UPA5**, UPA0, selmux, **ACCSEC**	–
`1C1`	$C, N, V, Z \leftarrow C, N, V, Z_{UPA}$	nOEUPA=0, nDUPA=0, CC, CN, CV, CZ	`01111/1/11/000`
`1C2`	Reg. Dir ← PC	nEPC2=0, nCBD=0	`11000/0/01/009`

2.4.3. JE (EXT), opcode `7D`

Para la instrucción `JE` salta a la dirección de 16 bits que sigue al opcode cuando los operandos comparados son iguales ($Z = 1$). Se copió la rutina de `JMP` (`7E0`–`7E5`) y se modificó sólo el

campo de secuenciamiento de `7D3`, se seleccionó la prueba `10010` (Z), con $VF = 0$, instrucción `01` y liga `7D5`. De forma que la palabra completa de `7D3` queda de la siguiente manera:

$$\underbrace{\texttt{10010}}_{Z}\ \underbrace{\texttt{0}}_{VF}\ \underbrace{\texttt{01}}_{\text{salto}}\ \underbrace{\texttt{0111 1101 0101}}_{\texttt{7D5}}\ \underbrace{\texttt{1001001000...0010}}_{\text{salidas iguales a } \texttt{7E3}}.$$

Si $Z = 0$, la entrada coincide con VF, $CC = 1$ y el secuenciador carga la liga `7D5`, omitiendo `7D4` por lo que el PC conserva la dirección siguiente al `JE`. Si $Z = 1$, $CC = 0$ y se continúa en `7D4`, donde se ejecuta PC ← RA. Las salidas de `7D3` no cambian, por lo que el byte bajo de la dirección se lee en ambos casos y el PC siempre avanza tres bytes. La Figura [4] muestra las cartas ASM resultantes para `CMP` y `JE`.

Tabla 4: Microprograma de `JE` (EXT) en `7D0`–`7D5`.

Dir.	Transferencia	Señales activas	Sec.
`7D0`	Reg. Dir ← PC	nEPC2=0, nCBD=0	–
`7D1`	RA_{ALTA} ← Mem, PC ← PC+1	nERA1=0, RA2, PC0, nAS=0, BD	–
`7D2`	Reg. Dir ← PC	nEPC2=0, nCBD=0	–
`7D3`	RA_{BAJA} ← Mem, PC ← PC+1	nERA0=0, RA1, RA0, PC0, nAS=0	`10010/0/01/7D5`
`7D4`	PC ← RA	nERA1=0, nERA0=0, nEPC1=0, nEPC0=0, PC2, PC0	`01111/1/11/000`
`7D5`	Reg. Dir ← PC	nEPC2=0, nCBD=0	`11000/0/01/009`

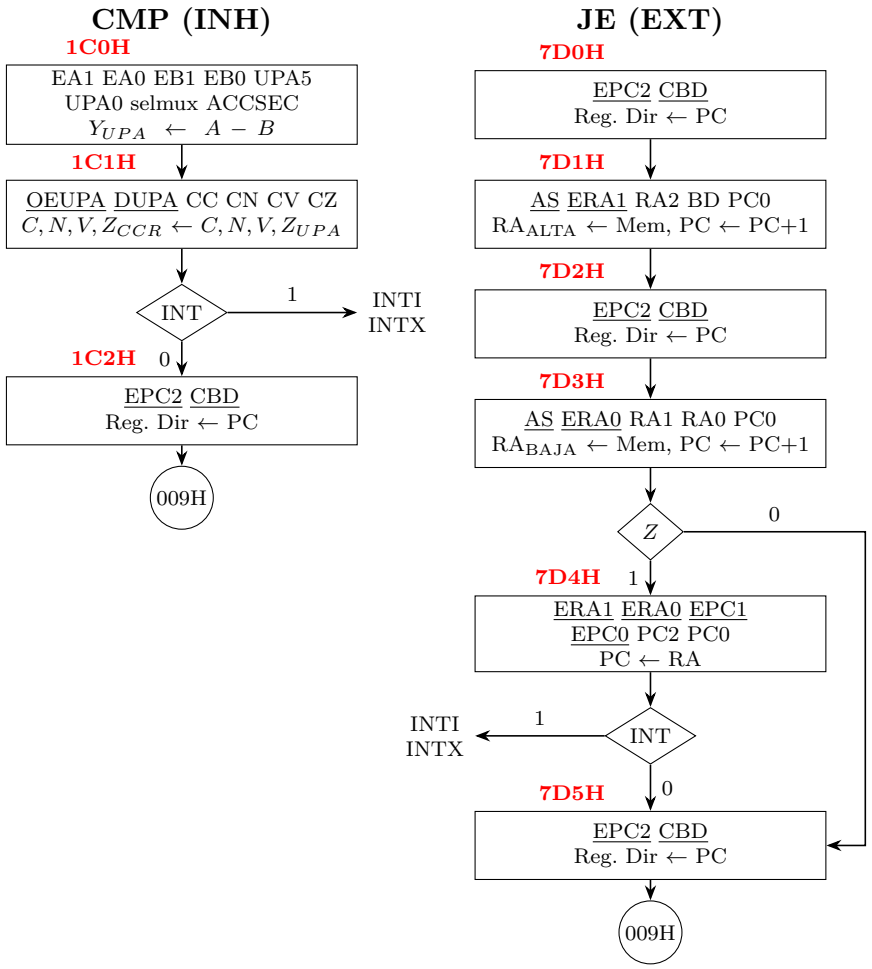


Figura 4: Cartas ASM de las instrucciones `CMP` (INH) y `JE` (EXT).
NOTA: Las señales subrayadas son activas en bajo.

2.4.4. STAA (EXT), opcode `A7`

En `STAA` se implementó directamente a partir de la carta ASM de la Figura[3]. Los estados `A70`–`A73` son iguales a `7E0`–`7E3`. En lugar de copiar RA al PC, en `A74` se carga el registro de direcciones con RA ($\overline{\text{ERA2}}$, $\overline{\text{CBD}}$) y en `A75` se escribe A en la memoria ($\overline{\text{AS}}$, $\overline{\text{R/W}}$, EA0). El estado `A76` actualiza Z y N con el valor de A y limpia V, por lo que se copió la fila `862`. Finalmente, `A77` es igual a `863`.

Tabla 5: Microprograma de `STAA` (EXT) en `A70`–`A77`.

Dir.	Transferencia	Señales activas	Sec.
`A70`–`A73`	Lectura de la dirección en RA	Iguales a `7E0`–`7E3`	–
`A74`	Reg. Dir ← RA	nERA2=0, nCBD=0	–
`A75`	Mem ← A	EA0=1, nAS=0, nRW=0	–
`A76`	$V \leftarrow 0$; $Z, N \leftarrow Z_A, N_A$	B6, B3, B2, CN, CV, CZ	`01111/1/11/000`
`A77`	Reg. Dir ← PC	nEPC2=0, nCBD=0	`11000/0/01/009`

2.4.5. LDAA (EXT), opcode `96`

La carga extendida de A tiene la misma secuencia que `STAA`, excepto que la memoria se lee en lugar de escribirse. Por ello se copió la rutina `A70`–`A77` y sólo se modificó el sexto estado: en `965` se regresó `nRW` a 1 (lectura) y se activó `nWA=0`, manteniendo `EA0=1` y `nAS=0`. Así se obtiene la transferencia $A \leftarrow$ Mem.

2.4.6. LDAB (EXT), opcode `F6`

Para la carga extendida de B se copió la rutina de `LDAA` (EXT) y en `F65` se cambiaron las señales del acumulador: se desactivaron `EA0` y `nWA` y se activaron `EB0=1` y `nWB=0`, para obtener $B \leftarrow$ Mem. El estado `F66` se conservó igual a `966`, por lo que evalúa Z y N con el contenido de A (bits $B6$ y $B3$) y no con el de B.

Tabla 6: Diferencias entre `STAA`, `LDAA` y `LDAB` en modo extendido. Los demás estados son iguales.

Instrucción	Sexto estado	Transferencia	Señales activas
`STAA` (EXT)	`A75`	Mem ← A	EA0=1, nAS=0, **nRW=0**
`LDAA` (EXT)	`965`	$A \leftarrow$ Mem	EA0=1, nAS=0, **nWA=0**
`LDAB` (EXT)	`F65`	$B \leftarrow$ Mem	**EB0=1**, nAS=0, **nWB=0**

2.5. Módulos implementados en VHDL

A diferencia de las prácticas anteriores, la arquitectura base del procesador ya está descrita en el proyecto, por lo que únicamente se modificó la memoria de microprograma dentro del secuenciador.

Memoria de microprograma (`memory.vhd`). Almacena las palabras de control de 98 bits correspondientes a cada microdirección del secuenciador. El Código [1] muestra las rutinas agregadas en `memory.vhd`.

Código 1: Microinstrucciones agregadas en `memory.vhd`.

```
    --LDAB (INM)
    elsif(dir= X"D60") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"D61") then data <= "00000000000000000000101000100000000000011011100001110001110001110000111111010000000000000000000010";
    elsif(dir= X"D62") then data <= "01111111000000000000100100100000000000011011100001110001110001110001111111000000010010100011000010";
    elsif(dir= X"D63") then data <= "11000001000000001001100100100000000000011011100001110001110000110000111111000000000000000000000010";

    --CMP (INH)
    elsif(dir= X"1C0") then data <= "00000000000000000000111111100000100001111111000011100011100011100011111111000000000000000000000011";
    elsif(dir= X"1C1") then data <= "01111111000000000000100100100000000000000111000011100011100011100011111111000000000000000011110000010";
    elsif(dir= X"1C2") then data <= "11000001000000001001100100100000000000011011100001110001110000110000111111000000000000000000000010";

    --JE (EXT)
    elsif(dir= X"7D0") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"7D1") then data <= "00000000000000000000100100100000000000011011100001011001110001110011111111010000000000000000000010";
    elsif(dir= X"7D2") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"7D3") then data <= "10010001011111010101100100100000000000011011100001100111110001110011111110100000000000000000000010";
    elsif(dir= X"7D4") then data <= "01111111000000000000100100100000000000011011100001000001110001001011111111000000000000000000000010";
    elsif(dir= X"7D5") then data <= "11000001000000001001100100100000000000011011100001110001110000110000111111000000000000000000000010";

    --STA (EXT)
    elsif(dir= X"A70") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"A71") then data <= "00000000000000000000100100100000000000011011100001011001110001110011111111010000000000000000000010";
    elsif(dir= X"A72") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"A73") then data <= "00000000000000000000100100100000000000011011100001100111110001110011111110100000000000000000000010";
    elsif(dir= X"A74") then data <= "00000000000000000000100100100000000000011011100000110001110001110000111111000000000000000000000010";
    elsif(dir= X"A75") then data <= "00000000000000000000100101100000000000011011100001110001110001110001111110000000000000000000000010";
    elsif(dir= X"A76") then data <= "01111111000000000000100100100000000000011011100001110001110001110001111111000000001001100011000010";
    elsif(dir= X"A77") then data <= "11000001000000001001100100100000000000011011100001110001110000110000111111000000000000000000000010";

    -- LDAA (EXT)
    elsif(dir= X"960") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"961") then data <= "00000000000000000000100100100000000000011011100001011001110001110011111110100000000000000000000010";
    elsif(dir= X"962") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"963") then data <= "00000000000000000000100100100000000000011011100001100111110001110011111110100000000000000000000010";
    elsif(dir= X"964") then data <= "00000000000000000000100100100000000000011011100000110001110001110000111111000000000000000000000010";
    elsif(dir= X"965") then data <= "00000000000000000000100101000000000000011011100001110001110001110001111110100000000000000000000010";
    elsif(dir= X"966") then data <= "01111111000000000000100100100000000000011011100001110001110001110001111111000000001001100011000010";
    elsif(dir= X"967") then data <= "11000001000000001001100100100000000000011011100001110001110000110000111111000000000000000000000010";

    -- LDAB (EXT)
    elsif(dir= X"F60") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"F61") then data <= "00000000000000000000100100100000000000011011100001011001110001110011111110100000000000000000000010";
    elsif(dir= X"F62") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"F63") then data <= "00000000000000000000100100100000000000011011100001100111110001110011111110100000000000000000000010";
    elsif(dir= X"F64") then data <= "00000000000000000000100100100000000000011011100000110001110001110000111111000000000000000000000010";
    elsif(dir= X"F65") then data <= "00000000000000000000101000100000000000011011100001110001110001110001111110100000000000000000000010";
    elsif(dir= X"F66") then data <= "01111111000000000000100100100000000000011011100001110001110001110001111111000000001001100011000010";
    elsif(dir= X"F67") then data <= "11000001000000001001100100100000000000011011100001110001110000110000111111000000000000000000000010";
```

2.6. Programas en ensamblador

Los programas se escribieron en `mem_content.mif`, con `WIDTH=8`, `DEPTH=256`, direcciones en decimal (`ADDRESS_RADIX=UNS`) y datos en hexadecimal (`DATA_RADIX=HEX`). En los listados de ensamblador las direcciones se expresan en hexadecimal.

Durante la práctica se realizaron tres ejercicios, los cuales se describen a continuación.

2.6.1. Ejercicio 1

```
B = 0x02;
A = 0x00;
while (1) {
    A = A + B;
}
```

Código 2: Ejercicio 1 en ensamblador.

```
Dir.   Bytes      Instruccion
0000   D6 02      LDAB #$02       ; B <- 02
0002   86 00      LDAA #$00       ; A <- 00
0004   1B         ABA             ; A <- A + B
0005   7E 00 04   JMP  $0004      ; regresar a la suma
```

Código 3: Contenido de `mem_content.mif` para el ejercicio 1.

```
CONTENT BEGIN
    0  : D6;   -- LDAB inm
    1  : 02;
    2  : 86;   -- LDAA inm
    3  : 00;
    4  : 1B;   -- ABA inh
    5  : 7E;   -- JMP ext
    6  : 00;
    7  : 04;   -- 04: ciclo de suma; 08: salto a LDAA #FF
    8  : 86;   -- LDAA inm
    9  : FF;
    [10..255] : 00;
END;
```

Para implementar el ciclo del pseudocódigo se agregó la rutina de `LDAB (INM)` y se tradujo la secuencia a ensamblador. Primero se cargan $B = 02$ y $A = 00$; después, `ABA` suma ambos acumuladores y `JMP $0004` regresa a la instrucción `ABA`, por lo que la suma se repite indefinidamente. En el archivo `mem_content.mif`, la dirección `0007` puede configurarse, en este caso el número es `4`.

2.6.2. Ejercicio 2

```
B = 0x05;
A = valor;          // 0x05 (iguales) o 0x09 (diferentes)
if (A != B) {
    A = 0x48;
}
A = 0x63;
```

La condición $A \neq B$ se implementa con `CMP (INH)` y `JE (EXT)` de formma quie fue necesaerio implementar estas rutinas. En el programa, si los acumuladores son iguales, `JE` salta directamente a la asignación final; si son diferentes, el programa continúa con la asignación de `48`.

Código 4: Ejercicio 2 en ensamblador.

```
Dir.   Bytes      Instruccion
0000   D6 05      LDAB #$05      ; B <- 05
0002   86 09      LDAA #$09      ; A <- 09 (05 para la prueba de igualdad)
0004   1C         CMP            ; A - B, actualiza Z
0005   7D 00 0A   JE   $000A     ; si A = B, saltar a FIN
0008   86 48      LDAA #$48      ; A <- 48 (solo si A != B)
000A   86 63      LDAA #$63      ; FIN: A <- 63
```

Código 5: Contenido de `mem_content.mif` para el ejercicio 2.

```
CONTENT BEGIN
    0  : D6;   -- LDAB inm
    1  : 05;
    2  : 86;   -- LDAA inm
    3  : 09;   -- 05 para la prueba de igualdad
    4  : 1C;   -- CMP inh (A - B)
    5  : 7D;   -- JE ext
    6  : 00;
```

```
    7  : 0A;
    8  : 86;    -- LDAA inm
    9  : 48;
    10 : 86;    -- LDAA inm
    11 : 63;
    [12..255] : 00;
END;
```

Para este ejercicio se cargó B con `05` y A con `09`; para probar la igualdad basta cambiar el dato de la dirección decimal 3 a `05`. La instrucción `CMP` calcula $A - B$ y actualiza la bandera Z. Si los acumuladores son iguales, $Z = 1$ y `JE $000A` salta directamente a la asignación final, evitando la instrucción ubicada en `0008`. Si son diferentes, $Z = 0$, el salto no se toma y `LDAA #$48` se ejecuta antes de cargar `63` en A. Así, el resultado final siempre es $A =$ `63`, pero el valor intermedio `48` sólo aparece en la prueba de desigualdad.

2.6.3. Ejercicio 3

Para este ejercicio las variables `temp` y `res` se encuentran en la RAM, en las direcciones `0017` y `0018` (posiciones 23 y 24). Por ello se emplean las cargas y el almacenamiento en modo extendido.

```
temp = 0x0A;
res  = valor;       // 0x0A (iguales) o 0x1B (diferentes)
if (res != temp) {
    res = 0x48;
}
res = 0x63;
while (1) { }
```

Código 6: Ejercicio 3 en ensamblador.

```
Dir.   Bytes      Instruccion
0000   F6 00 17   LDAB $0017     ; B <- temp
0003   96 00 18   LDAA $0018     ; A <- res
0006   1C         CMP            ; A - B, actualiza Z
0007   7D 00 0F   JE   $000F     ; si res = temp, saltar a FIN
000A   86 48      LDAA #$48      ; A <- 48
000C   A7 00 18   STAA $0018     ; res <- 48
000F   86 63      LDAA #$63      ; FIN: A <- 63
0011   A7 00 18   STAA $0018     ; res <- 63
0014   7E 00 14   JMP  $0014     ; ciclo final
0017   0A                       ; temp
0018   1B                       ; res (0A para la prueba de igualdad)
```

Código 7: Contenido de `mem_content.mif` para el ejercicio 3.

```
CONTENT BEGIN
    0  : F6;    -- LDAB ext
    1  : 00;
    2  : 17;    -- temp (0x17 -> 23)
    3  : 96;    -- LDAA ext
    4  : 00;
    5  : 18;    -- res (0x18 -> 24)
    6  : 1C;    -- CMP inh (A - B)
    7  : 7D;    -- JE ext
    8  : 00;
```

```
    9  : 0F;
    10 : 86;    -- LDAA inm
    11 : 48;
    12 : A7;    -- STAA ext
    13 : 00;
    14 : 18;
    15 : 86;    -- LDAA inm
    16 : 63;
    17 : A7;    -- STAA ext
    18 : 00;
    19 : 18;
    20 : 7E;    -- JMP ext a 0x14
    21 : 00;
    22 : 14;
    23 : 0A;    -- temp
    24 : 1B;    -- res (0A para la prueba de igualdad)
    [25..255] : 00;
END;
```

Para este ejercicio se agregaron las rutinas de `LDAB (EXT)`, `LDAA (EXT)` y `STAA (EXT)`, debido a que los operandos se almacenan en memoria y no forman parte del flujo inmediato de instrucciones. Primero, `LDAB $0017` carga en B el valor de `temp`, mientras que `LDAA $0018` carga en A el valor de `res`. Después, `CMP` compara ambos acumuladores. Si son iguales, `JE $000F` salta directamente a la asignación de `63`, en caso de ser diferentes, se carga `48` en A y `STAA $0018` lo almacena temporalmente en `res`. En ambos casos se ejecuta después `LDAA #$63` y se guarda `63` en `res` mediante `STAA $0018`. Finalmente, `JMP $0014` salta a sí misma para mantener el programa en un ciclo infinito. En el archivo MIF, las direcciones `0017` y `0018` corresponden a las posiciones decimales 23 y 24, respectivamente.

3. Simulación

Para la ejecución de las simulaciones funcionales se realizaron en Quartu (ModelSim) con un reloj de 10 [ns] de periodo, `RESET=1` e `IRQn=XIRQn=1`, es decir, sin solicitudes de interrupción. En las salidas se observan los acumuladores (`ACCA_D`, `ACCB_D`), el registro auxiliar (`AUX_D`), el contador de programa (`PC_D`), la habilitación de la RAM (`csnn`, activa en bajo), la bandera Z y el estado presente del secuenciador (`Edo_Pres`).

Cada estado presente dura un ciclo de reloj (10 ns), las señales de control generadas en un estado se reflejan en la ruta de datos durante el ciclo siguiente.

3.1. Ejercicio 1

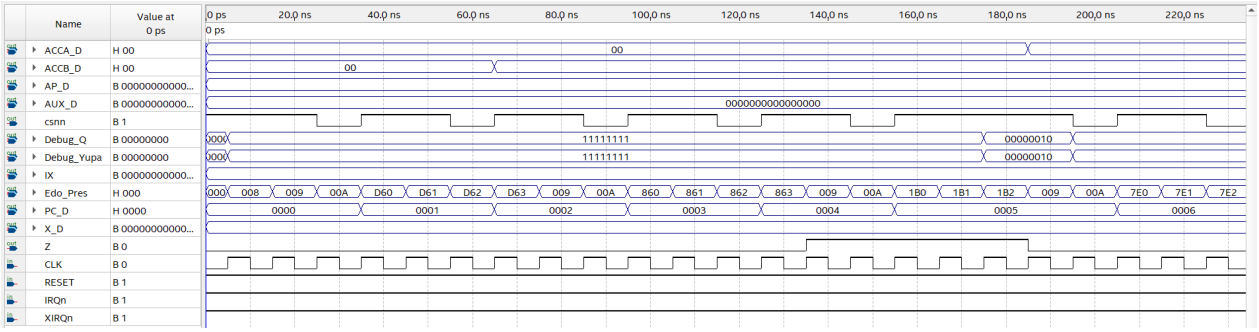


Figura 5: Simulación del ejercicio 1 con salto a `0004`.

Al inicio de la gráfica obtenida de la simulación en la Figura [5], el secuenciador (`Edo_Pres`) inicia en `000`, cuya liga lleva a `008`, después recorre `009` y `00A`, y el PC pasa de `0000` a `0001`. La instrucción MAP lleva el opcode `D6` a `D60`. Durante `D60`–`D63` se lee el operando y B cambia a `02`. El segundo grupo de cuatro estados corresponde a `860`–`863`: A se carga con `00` y, como el resultado es cero, Z cambia a 1 tras ejecutarse `862`.

Posteriormente aparecen tres estados `B` (`1B0`–`1B2`). Al concluir `1B1`, la salida de la UPA se escribe en A, que cambia a `02`, y Z regresa a 0. Después siguen seis estados `E` (`7E0`–`7E5`): en `7E1` y `7E3` se leen los bytes `00` y `04`, respectivamente. Durante `7E3`, el PC se incrementa para avanzar a la siguiente posición y muestra `0008` de forma transitoria, de forma que `AUX_D` toma el valor `0004` al capturar el segundo byte. Finalmente, en `7E4` se ordena PC ← RA y el PC queda en `0004`, que es la dirección destino del salto.

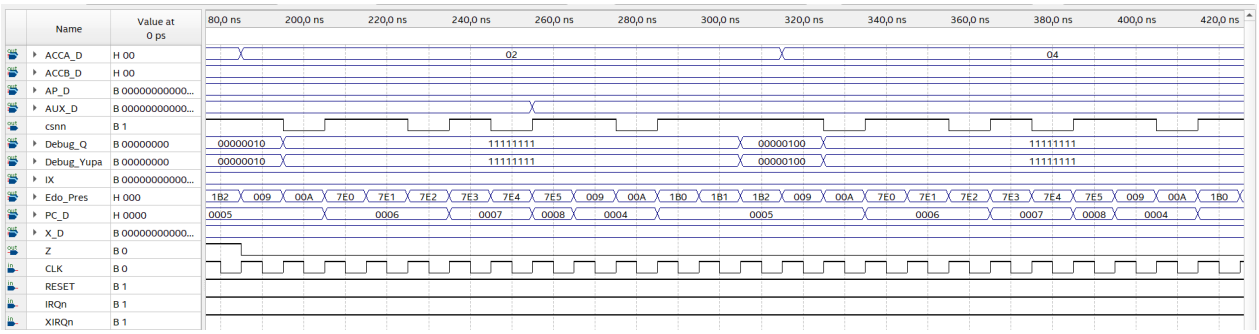


Figura 6: Simulación del ejercicio 1 con salto a `0004` - Parte 2.

A partir de ese punto la secuencia de estados presentes se repite: `009`, `00A`, `1B0`–`1B2`, `009`, `00A`, `7E0`–`7E5`. Son 13 estados por iteración En el PC se repite el patrón `0005`, `0006`, `0007`, `0008`, `0004`, y B permanece en `02`.

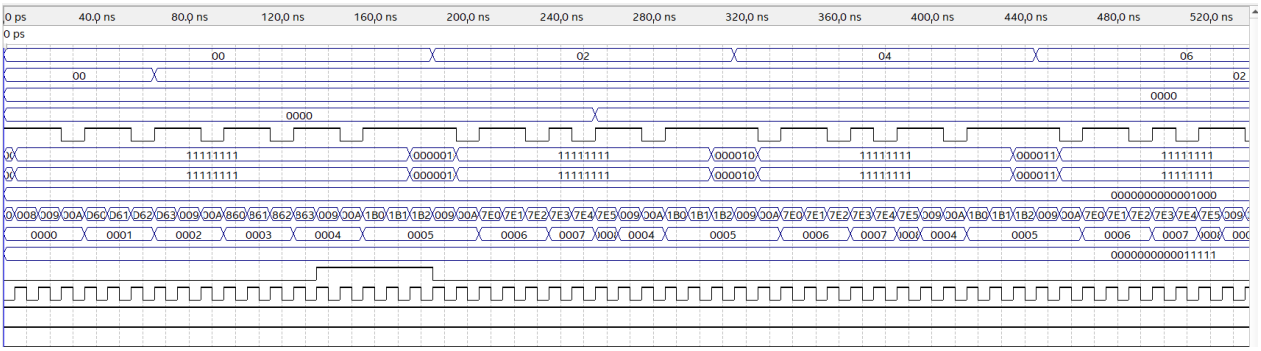


Figura 7: Simulación del ejercicio 1 con salto a `0004` - Parte 3.

En la Figura [8] la secuencia es idéntica hasta el primer `JMP`, pero ahora `AUX_D` toma `0008` en y el PC salta a esa dirección. Después del salto aparece un grupo de cuatro estados (`860`–`863`) y A cambia de `02` a `FF`. Desde `000A` la memoria contiene `00`, la instrucción MAP lleva ese opcode a la microdirección `000`, cuya liga regresa a `008`. Por ello `Edo_Pres` sólo muestra estados (`000`, `008`, `009`, `00A`) y el PC aumenta en uno cada cuatro ciclos, sin modificar los acumuladores.

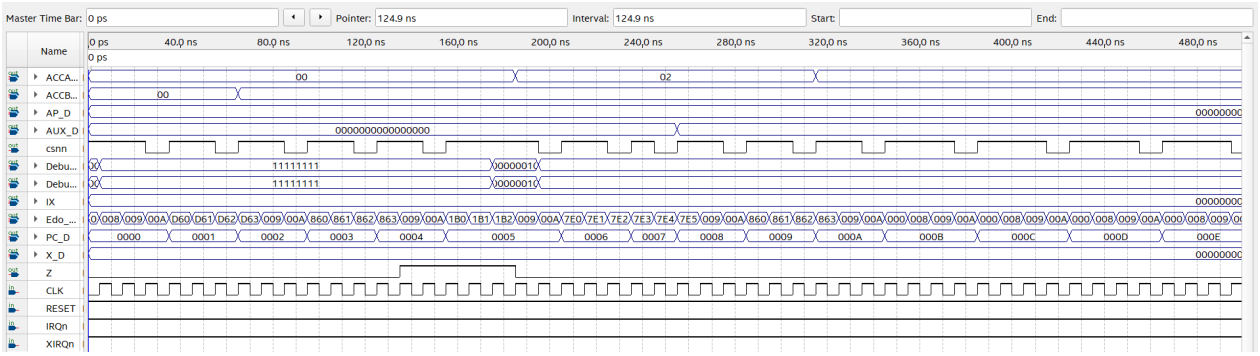


Figura 8: Simulación del ejercicio 1 con salto a `0008` - Parte 1.

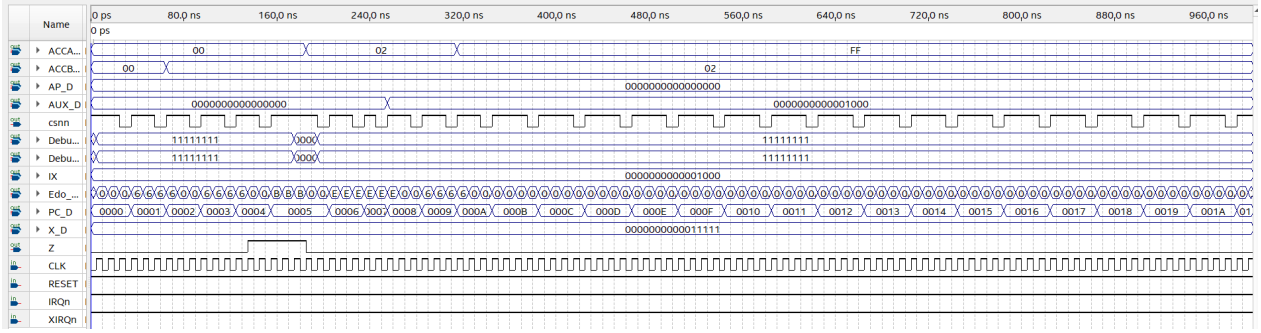


Figura 9: Simulación del ejercicio 1 con salto a `0008` - Parte 2.

3.2. Ejercicio 2

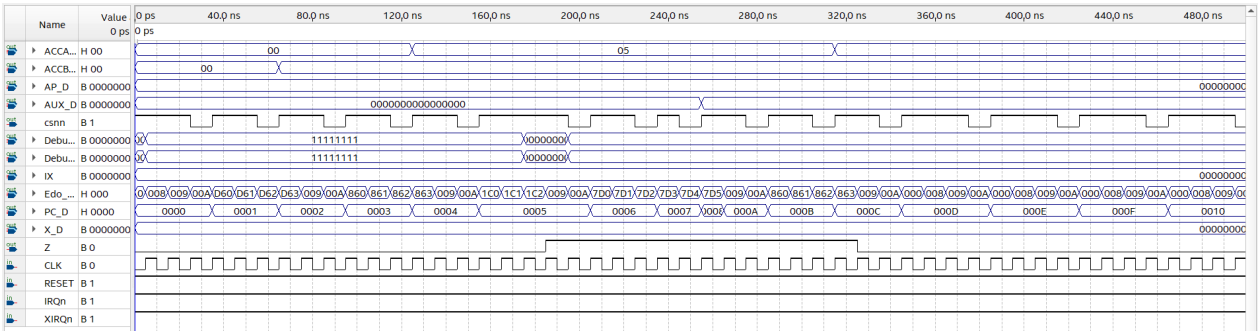


Figura 10: Simulación del ejercicio 2 con $A = B =$ 05. - Parte 1

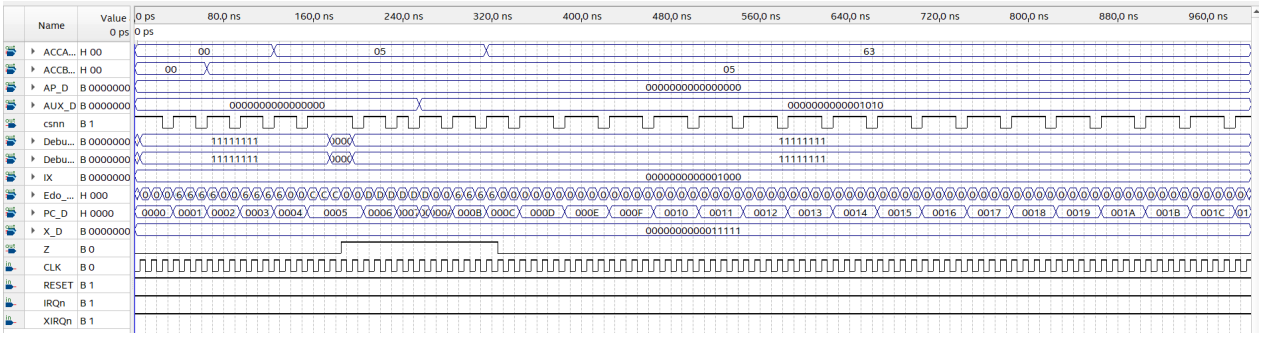


Figura 11: Simulación del ejercicio 2 con $A = B =$ 05. - Parte 2

En la Figura[10] los dos primeros grupos de estados cargan $B =$ 05 y $A =$ 05. Después aparecen tres estados C (1C0–1C2), donde en 1C1 se actualizan las banderas con $A - B = 0$ y Z cambia a 1, mientras A conserva 05.

El grupo siguiente tiene seis estados D: 7D0–7D5. En 7D3 se prueba Z; como $Z = 1$ y $VF = 0$, $CC = 0$ y el secuenciador continúa en 7D4, donde el PC se carga con AUX_D= 000A. En PC_D se observa la secuencia 0006, 0007, 0008, 000A: la dirección 0008 aparece sólo durante un ciclo por el incremento efectuado en 7D3, pero 7D4 se sobrescribe de inmediato con 000A, de modo que la instrucción en 0008 nunca se busca ni se ejecuta. Finalmente se ejecuta 860–863, A cambia directamente de 05 a 63 y Z regresa a 0, debido a que el dato cargado es distinto de cero.

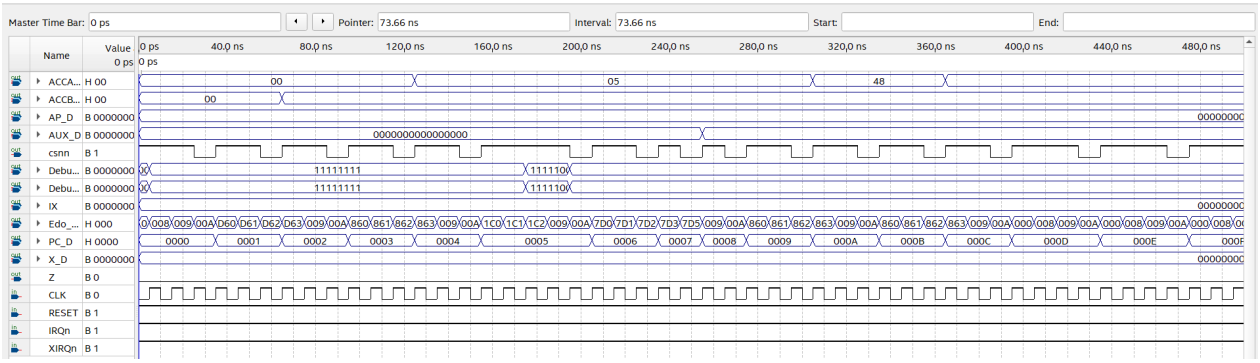


Figura 12: Simulación del ejercicio 2 con $A =$ `09` y $B =$ `05` - Parte 1.

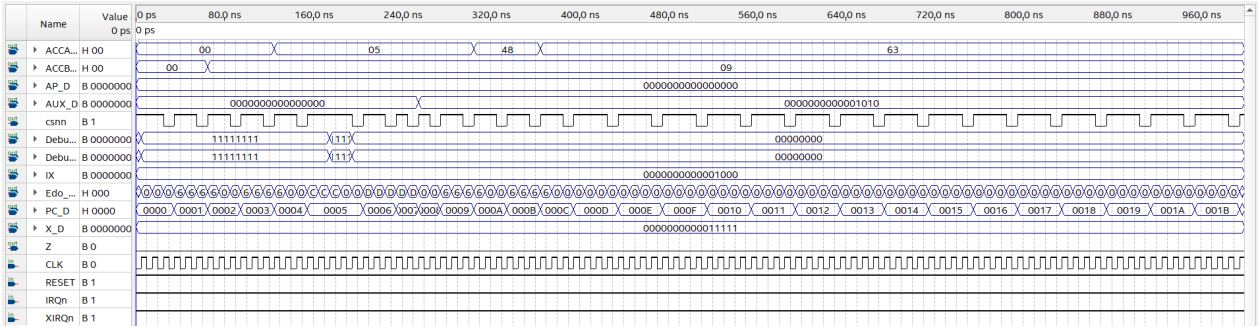


Figura 13: Simulación del ejercicio 2 con $A =$ `09` y $B =$ `05` - Parte 2.

En la Figura [12] se carga $A =$ `09`. La comparación produce $A - B =$ `04`, por lo que Z permanece en 0 durante toda la simulación. En este caso el grupo de estados `D` tiene sólo cinco estados: en `7D3`, $Z = 0$ coincide con $VF = 0$, $CC = 1$ y se carga la liga `7D5`, omitiendo `7D4`. Aunque `AUX_D` también recibe `000A`, esa dirección no se copia al PC, que continúa secuencialmente en `0008` y `0009`.

Después del `JE` (`7D`) se observan dos grupos de cuatro estados `86X`: el primero carga `48` y el segundo `63`. El valor final de A es `63` en los dos casos, sin embargo la diferencia principal entre ellos se identifica por la rama del secuenciador, el recorrido del PC y el valor intermedio `48`.

3.3. Ejercicio 3

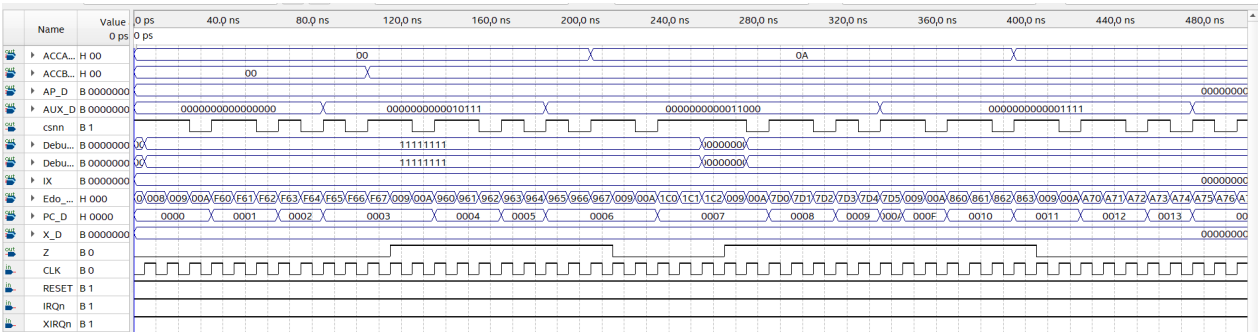


Figura 14: Simulación del ejercicio 3 con `temp`=`res`= `0A`. - Parte 1

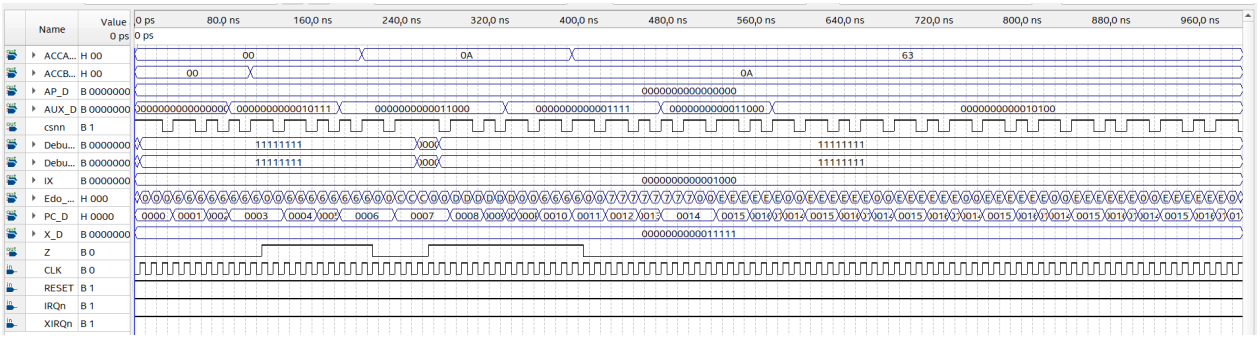


Figura 15: Simulación del ejercicio 3 con `temp`=`res`= `0A`. - Parte 2

En la Figura [14] el primer grupo de ocho estados corresponde a `F60`–`F67`. En `F61` y `F63` se leen los bytes de dirección y `AUX_D` toma `0017`, después en `F64` el registro de direcciones se carga con RA y en `F65` el dato `0A` se escribe en B. En `F66`, Z cambia a 1, aunque $B =$ `0A`: como este estado se copió de `966`, las banderas se calculan con A, que todavía vale `00`. La corrección consiste en usar $B7$ y $B4$ en lugar de $B6$ y $B3$, como en `D62`. Este detalle no afecta al programa, porque Z se vuelve a calcular en `CMP` antes de usarse.

El segundo grupo de ocho estados (`960`–`967`) lee la dirección `0018` (`AUX_D`= `0018`) y carga $A =$ `0A`; Z regresa a 0. En los tres estados `C` (`960`–`962`) la comparación da cero y Z cambia a 1. Por ello el grupo de estados `D` tiene seis estados (`7D0` - `7D5`): se ejecuta `7D4` y el PC salta a `000F` (tras el valor transitorio `0010` de `7D3`), a la vez que `AUX_D` contiene `000F`. Se ejecuta `LDAA #$63` ($A =$ `63`, $Z = 0$) y luego un solo grupo de ocho estados `7` (`A70`–`A77`), en el que `AUX_D` vuelve a `0018`y en `A75` se habilita la escritura (`csnn=0`) para guardar `63` en `res`.

Finalmente aparece el ciclo de `JMP`: `AUX_D` toma `0014` y `Edo_Pres` repite `7E0`–`7E5`, `009`, `00A`, mientras el PC recorre `0015`, `0016`, `0017`, `0014`. La simulación no muestra directamente el

contenido interno de la RAM por lo que la escritura se verifica por la ejecución de `A75`, el pulso de `csnn`, la dirección en RA y el valor de A.

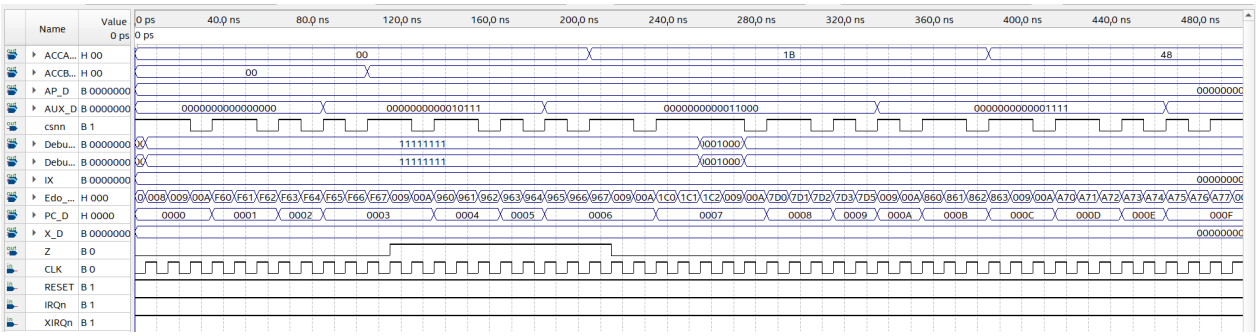


Figura 16: Simulación del ejercicio 3 con `temp`= `0A` y `res`= `1B` - Parte 1

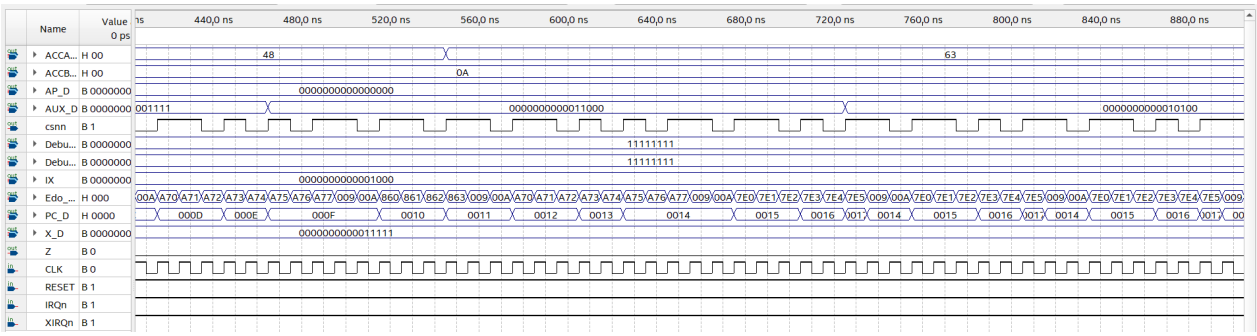


Figura 17: Simulación del ejercicio 3 con `temp`= `0A` y `res`= `1B` - Parte 2

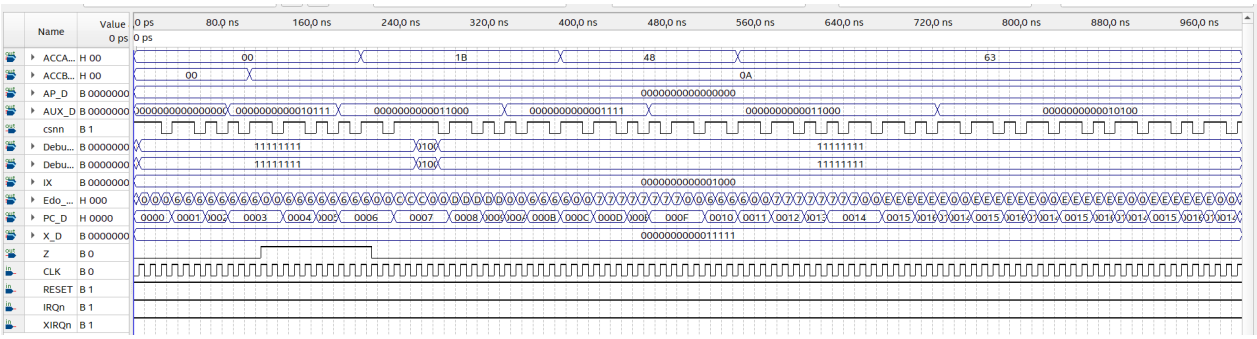


Figura 18: Simulación del ejercicio 3 con `temp`= `0A` y `res`= `1B` - Parte 3

En la Figura [16] las cargas extendidas producen $B =$ `0A` (105 ns) y $A =$ `1B` . La bandera Z se activa en `F66` por la misma razón descrita en la prueba anterior y regresa a 0 al cargar $A =$ `1B`, posteriormente de `CMP` se mantiene en 0 porque `1B` $-$ `0A` $\neq 0$. El grupo de estados

D tiene cinco estados, donde en 7D3 se carga la liga 7D5 y el PC continúa en 000A, aunque AUX_D contiene 000F.

A diferencia del caso anterior, se observan dos pares de rutinas LDAA/STAA: un grupo de cuatro estados (860-863) carga $A = 48$, un grupo de ocho estados (A70 - A77) lo guarda en 0018 (csnn=0). Posteriormente otro grupo de cuatro estados (860-863) carga $A = 63$ y un segundo grupo de estados (A70 - A77)lo escribe en la misma dirección (csnn=0). Por ello AUX_D permanece en 0018 durante ambos almacenamientos. Al final, el programa entra al mismo ciclo de JMP a 0014.

4. Conclusión

Percival Ulises Espinoza Matamoros. Gracias a la reealizacipon de está práctica pude poner en práctica conocimientos adquirodos en prácticas anteriores, sin embargo al implementar rutinas como `LDAB (INM)`, `CMP (INH)` y `JE (EXT)`, `STAA(EXT)` y `LDAA (EXT)` a partir de las cartas ASM proporcionadas así como en alguinos casos a partir de rutinas ya implementadas, me permitió comprender mejor el funcionamiento del procesador, además al usar las rutinas que se implementaron para escribir código en ensamblador me permitió tener una mejor comprensión de la forma en que se ejecutan las instrucciones desde el alto nivel hasta el microprograma, así como la forma en que se ejecutan las instrucciones y cómo se modifican los registros y las banderas de estado. Por otro lado en la simulación de los programas en ensamblador se pudo observar cómo se ejecutan las instrucciones analizando el estado presente del secuenciador, el contenido de los registros y las banderas de estado comprobando así el funcionamiento de los programas y la correcta implementación de las rutinas.

Oscar Iván Montiel Juárez. En esta practica entendimos de manera práctica como activar las lineas de control de nuestro procesador, además de entender la arquitectura completa del procesador (sus buses de datos, registros, acumuladores, etc), para la parte práctica además realizamos dos cartas asm (implementamos su funcionameinto) por lo que tuvimos que cambiar la información de la memoria (meminfo y memoria, es decir la RAM y la ROM) y finalmente comprobamos en la simulación que los registros se comportaran de la manera adecuada, he de mencionar tambien que implementamos instrucciones de salto y de comaración.

Amy Valentina Veraza García. La implementación de `CMP` y `JE` ayudó a comprender cómo se construye una decisión en el procesador: `CMP` sólo actualiza las banderas y `JE` utiliza Z mediante la prueba, el bit VF y la liga del secuenciador. Con el modo extendido fue posible trabajar con variables almacenadas en memoria. También se observó que copiar una rutina sin revisar todas sus salidas puede producir efectos no deseados, como la actualización de Z con el acumulador A en `LDAB` (EXT).